Author
Ayush Choudhary
23f3000370
23f3000370@ds.study.iitm.ac.in
I'm from saharanpur

Description
This project is a **Flask-based Quiz Application** that allows **users** to attempt quizzes and **admins** to manage subjects, chapters, and quiz questions. It includes **user authentication**, **quiz attempts tracking**, and **score summaries** displayed using **charts**.

Technologies Used

- **Flask** – Web framework for handling routes and views
- **SQLite** – Lightweight database for storing quiz data
- **Flask-Session** – Managing user sessions
- **Jinja2** – Template engine for rendering dynamic HTML pages
- **Chart.js** – Visualization library for summary charts
- **Flask Flash** – Displays feedback messages to users (e.g., success/error notifications)

Purpose of Technologies:

- Flask simplifies backend development and routing.
- SQLite provides an embedded, easy-to-manage database.
- Chart.js helps visualize quiz statistics interactively.
- Flask Flash provides real-time feedback to users for actions like adding quizzes or handling errors.

DB Schema Design

Tables

1. **SUBJECTS** (`id`, `subject_name`)
    - Stores quiz subjects.
2. **CHAPTERS** (`id`, `chapter_name`, `subject_id`, `no_of_questions`)
    - `subject_id` as **Foreign Key** referencing `SUBJECTS(id)`.
3. **QUIZES** (`id`, `quiz_name`, `chapter_id`, `date`, `duration`)
    - `chapter_id` as **Foreign Key** referencing `CHAPTERS(id)`.
4. **QUESTIONS** (`id`, `quiz_id`, `question`, `option1`, `option2`, `option3`, `option4`, `answer`)
    - `quiz_id` as **Foreign Key** referencing `QUIZES(id)`.
5. **USERS** (`id`, `username`, `password`, `full name`, `qualification`, `dob`)
    - Role: **user**
6. **QUIZ_ATTEMPTS** (`id`, `user_id`, `quiz_id`, `score`, `attempt_date`)
    - Tracks user quiz attempts.
7. **ADMIN** (`id`, `username`, `password`)

- Predefined username and password for admin role

Schema Design Reasoning:

- **Foreign keys** ensure data consistency.
- **Separate tables** for modularity (subjects → chapters → quizzes → questions).

API Design

- **User Authentication:** `/login`, `/logout`, `/signup`, `/admin_login`
- **Subject Management:** `/add_subject`, `/edit_subject`, `/delete_subject`
- **Chapter Management:** `/add_chapter`, `/edit_chapter`, `/delete_chapter`
- **Summary:** `/admin_summary`, `/user_summary`
- **Quiz Management:** `/add_quiz`, `/edit_quiz`, `/delete_quiz/view_quiz`
- **Question Management:** `/add_question`, `/edit_question`, `/delete_question`
- **Quiz Attempts:** `/attempt_quiz`, `/submit_quiz`, `/view_results`
- **Dashboard Stats:** `/admin_dashboard`, `/user_dashboard/search_admin`

Each endpoint interacts with the database using **SQLite (sqlite3 library)**, processes POST/GET requests, and returns JSON responses or HTML templates.

Architecture and Features

Project Structure:

```
/Quiz_Master_V1
├── /templates        # HTML templates
├── /static           # CSS, JS, images
├── /controllers      # Routes and API logic
├── /models           # Database models
├── app.py            # Main application
├── quiz_master.db    # SQLite database
```

Implemented Features:

✅ **User Authentication** – Secure login & role-based access

✅ **Quiz Management** – CRUD operations for subjects, chapters, quizzes, and questions

✅ **Quiz Attempt & Scoring** – Users can take quizzes, scores are stored

✅ **Dashboard with Charts** – Admin/User dashboards with **Chart.js visualizations**